

ME – 781

Assignment 3

Designing a Custom Loss Function

Submitted by: Soumitro Mukherjee

1. Description of the loss function

I propose a **Risk-Aware Asymmetric Relative Pseudo-Huber (RARPH) loss** for **real-time wind-turbine generator temperature prediction**. The problem is a regression task with noisy sensor observations, rare high-temperature events, and unequal consequences of under- and overprediction.

Let y_i be the measured temperature and $\hat{y}_i$ the prediction.

I. Define the relative residual:

$$r_i = \frac{\hat{y}_i - y_i}{|y_i| + \epsilon}, \quad \text{where } \epsilon > 0 \text{ prevents division by zero.}$$

II. Define an asymmetric penalty:

$$a(r_i) = \begin{cases} c_u, & r_i < 0 \\ c_o, & r_i \geq 0 \end{cases}$$

where

- c_u = penalty coefficient for underprediction,
- c_o = penalty coefficient for overprediction.

For the proposed application, $c_u > c_o$, because underprediction of a dangerous temperature is considered more costly.

III. Then, the risk-weighting function:

$$w(y_i) = 1 + \frac{\lambda}{1 + \exp[-k(y_i - T_w)]}$$

where:

- T_w = warning/critical point,
- λ = maximum additional risk weight,
- k = sharpness of the transition.

As y_i approaches and exceeds T_w , the loss receives greater weight.

IV. Finally, we use the pseudo-Huber function:

$$H_\delta(r) = \delta^2 \left[\sqrt{1 + \left(\frac{r}{\delta}\right)^2} - 1 \right]$$

where $\delta > 0$ controls the transition between quadratic and approximately linear behaviour.

The overall loss function: $L_i = w(y_i)a(r_i)H_\delta(r_i)$, and for n observations, $L_{\text{RARPH}} = \frac{1}{n} \sum_{i=1}^n L_i$.

2. How the Loss Function Works

RARPH measures prediction error relative to the actual value, penalizes underprediction more strongly than overprediction, gives greater importance to observations near a critical temperature, and uses a pseudo-Huber function to reduce the influence of extreme outliers.

Unlike MSE or MAE, it considers the size, direction, operating regime, and extremeness of the prediction error.

3. Scenario where MSE/MAE are misleading

Suppose the warning temperature is

$$T_w = 90^\circ C.$$

Consider:

$$y_1 = 95^\circ C, \hat{y}_1 = 85^\circ C$$

and

$$y_2 = 95^\circ C, \hat{y}_2 = 105^\circ C.$$

Both have the same absolute error:

$$|y_i - \hat{y}_i| = 10.$$

Hence MAE = 10 and MSE = 100 for both predictions.

But operationally they are not equivalent. Predicting $85^\circ C$ when the generator is actually at $95^\circ C$ may hide a dangerous high-temperature condition, whereas predicting $105^\circ C$ is conservative.

Let's assume,

$$\epsilon = 0.001, \delta = 0.1, c_u = 2, c_o = 1, \lambda = 1, k = 0.5.$$

For $y = 95$,

$$w(95) = 1 + \frac{1}{1 + e^{-0.5(95-90)}} \approx 1.92414.$$

For the underprediction $85^\circ C$,

$$r = \frac{85 - 95}{95 + 0.001} \approx -0.105262,$$

Since this is an underprediction,

$$H_\delta(-0.105) = 0.1^2 \left[\sqrt{1 + \left(\frac{-0.105262}{0.1} \right)^2} - 1 \right] \approx 0.004519.$$

Hence,

$$L_{\text{under}} = w(95) \times 2 \times 0.004519 = 1.92414 \times 2 \times 0.004519 \approx 0.01739$$

For the underprediction $105^\circ C$,

$$r = \frac{105 - 95}{95 + 0.001} \approx 0.105262,$$

Since this is an underprediction,

$$a(0.105) = c_o = 1.$$

The pseudo-Huber term is the same because it depends on r^2 .

Hence,

$$L_{\text{over}} = w(95) \times 1 \times 0.004519 = 1.92414 \times 1 \times 0.004519 \approx 0.008695$$

Thus, $L_{\text{under}} \approx 2L_{\text{over}}$ even though MSE and MAE regard the two errors as identical.

This is more appropriate when false reassurance is more costly than conservative prediction.

4. Differentiability, convexity, outliers and imbalance

I. Differentiability

The pseudo-Huber function is differentiable everywhere, $H'_\delta(r) = \frac{r}{\sqrt{1+(r/\delta)^2}}$.

At $r = 0$, $H'_\delta(0) = 0$, so the asymmetric weighting remains continuous in derivative at the origin. Hence RARPH is differentiable with respect to $\hat{y}$.

II. Convexity

Pseudo-Huber is convex. Since $r = \frac{\hat{y}-y}{|y|+\epsilon}$ is affine in $\hat{y}$, and all weights are positive, the loss is convex in $\hat{y}$.

For a nonlinear neural network $\hat{y} = f_\theta(x)$, however, the overall objective is not necessarily convex in the model parameters θ .

III. Outliers

MSE grows quadratically, $(y - \hat{y})^2$.

RARPH behaves approximately quadratically for small residuals but approximately linearly for large residuals:

$$H_\delta(r) \approx \frac{1}{2}r^2 \text{ for small } r,$$
$$H_\delta(r) \approx \delta |r| - \delta^2 \text{ for large } r.$$

Thus, isolated sensor spikes have much less influence than under MSE.

The loss is for regression, so ordinary class imbalance is not directly applicable. However, high-temperature observations may be rare. The risk weight $w(y) = 1 + \lambda \frac{1}{1+e^{-k(y-T_w)}}$ makes these rare but operationally important observations contribute more strongly.

Real-time implementation using Python:

I have developed the Risk-Aware Asymmetric Relative Pseudo-Huber Loss (RARPH) function using python's Math library. The function is then used to find the RARPH iteratively on a dummy dataset.

There are two python “.py” files named “[RARPH.py](#)” and “[Main.py](#)” (please refer to their hyperlinks). One should keep these two files under same directory, and then simply run the “Main.py” to examine the output.